

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

*I **Jannathul Firdaus. N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Jannathul Firdaus. N

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1: Intelligent Agents in a Smart Home Automation System

1.1 Introduction

An intelligent agent is anything that perceives its environment through sensors and acts upon it through actuators (Agent = Architecture + Program). A rational agent selects the action expected to maximise its performance measure, given its percept history and built-in knowledge. The smart home system is described using the PEAS framework:

PEAS	Smart home description
Performance	Occupant comfort, low energy consumption, security (fewer false alarms, no missed intrusions), user satisfaction
Environment	Rooms, occupants, weather, time of day, grid tariff; partially observable, stochastic, dynamic, continuous, sequential, multi-agent (humans)
Actuators	Lights, thermostat/AC/heater, door locks, alarm siren, camera, notifications
Sensors	Motion (PIR), temperature/humidity, light (LDR), door/window contacts, camera, clock, smartphone presence

1.2 Types of Intelligent Agents and their function in the smart home

(a) Simple Reflex Agent

It selects actions only from the current percept using condition–action rules (IF condition THEN action). It keeps no memory or model of the world, so it works properly only in a fully observable environment.

- **Smart home use:** IF motion detected AND room is dark THEN switch on light; IF door opened while alarm armed THEN sound alarm; IF temperature > 30°C THEN switch on fan.
- **Advantages:** Very fast, simple, cheap, reliable and easy to verify; ideal for safety-critical instant responses.
- **Limitations:** Cannot know if the room is occupied once the person stops moving (lights may switch off while someone reads); cannot learn, plan or trade off conflicting goals; may loop or give false alarms because it ignores history.

(b) Model-Based Reflex Agent

It maintains an internal state – a model of how the world evolves and how its own actions affect it – so it can handle partial observability. Action = rule(state), where state = update(previous state, last action, current percept).

- **Smart home use:** Remembers that a person entered the living room 5 minutes ago and has not left (door sensor history), so it keeps lights on even without motion; tracks which rooms are occupied, whether windows are open, and the trend of temperature (rising/falling).

- **Advantages:** Works with incomplete sensor data, infers hidden facts (occupancy, whether the house is empty), fewer false alarms.
- **Limitations:** Still reactive; has no explicit objective, so it cannot decide among alternative actions or plan ahead.

(c) Goal-Based Agent

In addition to a model, it has explicit goals and uses search and planning to choose action sequences that achieve them. Decisions are flexible because the goal, not a fixed rule, drives behaviour.

- **Smart home use:** Goal “room at 24°C by 7:00 pm when the user returns”: the agent plans to pre-cool at 6:30 pm. Goal “house secured by 11 pm”: it plans to check doors/windows, lock them, arm cameras and dim lights in sequence.
- **Advantages:** Adaptable, can plan multi-step tasks, new goals can be added without rewriting rules.
- **Limitations:** Goals are binary (achieved / not achieved); it cannot compare two ways of reaching the goal (a faster but more energy-hungry cooling vs. slower and cheaper). Planning costs computation.

(d) Utility-Based Agent

It uses a utility function $U(\text{state})$ that maps each state to a real number measuring “how happy” the agent is, and chooses the action with maximum expected utility: $a^* = \operatorname{argmax} \sum P(s'|a) \cdot U(s')$. It handles conflicting goals, uncertainty and trade-offs.

- **Smart home use:** $U = w_1 \cdot \text{Comfort} - w_2 \cdot \text{EnergyCost} + w_3 \cdot \text{Security} - w_4 \cdot \text{Annoyance}$. At peak tariff it pre-cools slightly less to save cost; with an unknown motion at night it raises security utility and sends an alert, even at the cost of some energy.
- **Advantages:** Gives optimal, rational decisions under uncertainty and multiple objectives; user preferences can be learned by tuning weights.
- **Limitations:** Designing a correct utility function and estimating probabilities is hard; computationally heavier.

1.3 Comparison

Feature	Simple reflex	Model-based	Goal-based	Utility-based
Uses memory/state	No	Yes	Yes	Yes
Handles partial observability	No	Yes	Yes	Yes
Has goals	No	No	Yes	Yes (via utility)
Handles trade-offs	No	No	No	Yes

Feature	Simple reflex	Model-based	Goal-based	Utility-based
Can learn preferences	No	Limited	Limited	Yes (with learning)
Speed / cost	Fastest / lowest	Fast	Moderate	Highest
Smart home example	Motion → light ON	Occupancy tracking	Pre-cool before arrival	Balance comfort vs cost

1.4 Analysis and Justification of the Best Approach

No single agent type satisfies all requirements. The system must (i) react instantly to real-time events, (ii) work with partial information, (iii) pursue goals such as a comfortable house at arrival time, (iv) balance comfort against energy cost and security, and (v) learn user habits over time. Therefore a hybrid layered architecture with a learning component is the most effective:

- **Layer 1 – Simple reflex (safety/security layer):** Hard-wired rules for emergencies: smoke/gas detected → alarm and unlock exits; forced entry → siren. These need minimum latency and must never depend on learning.
- **Layer 2 – Model-based layer:** Maintains the house state (who is where, which windows are open, temperature trend, time of day) to remove sensor ambiguity and prevent mistakes such as switching lights off on a still occupant.
- **Layer 3 – Goal-based planner:** Plans activities such as pre-heating, scheduled security arming and “away mode” routines.
- **Layer 4 – Utility-based decision maker:** Chooses among the possible plans by maximising the expected utility of comfort, energy efficiency and security, adjusting for tariff and weather.
- **Learning element:** Observes manual overrides (e.g. the user lowers the thermostat at night), uses a critic (user feedback, energy bills) and updates utility weights and rules, so the system personalises itself over time.

Justification: Reflex rules guarantee fast, dependable security; the model removes uncertainty; goals give proactive behaviour; and utility resolves conflicts between comfort and energy saving. Learning makes the agent improve with experience. A purely reflex design would be too rigid, and a purely utility-based design would be too slow for emergencies. Hence the hybrid learning agent with utility-based top-level decision making gives the best comfort, energy efficiency and security.

1.5 Conclusion

Simple reflex agents give speed, model-based agents give awareness, goal-based agents give purpose and utility-based agents give optimal trade-offs. Combining them in a layered, learning architecture produces a smart home that is responsive, efficient, secure and personalised.

Question 2: Maze-Solving Robot – Uninformed Search and Agent Design

2.1 Problem formulation

Component	Maze definition
States	Position (cell) of the robot in the maze
Initial state	Starting cell
Actions	Move Up / Down / Left / Right (if no wall)
Transition model	Result of moving to the adjacent cell
Goal test	Is the current cell the exit?
Path cost	Cost of each move (1 per step, or higher for turns, rough ground, etc.)

2.2 Uniform Cost Search (UCS)

Working: UCS expands the node with the lowest path cost $g(n)$ first using a priority queue (frontier). When a node is removed for expansion it is tested for the goal. If all step costs are equal, UCS behaves like Breadth-First Search (BFS). In the maze, the robot explores outward in “cost contours”, so the first time it reaches the exit it has used the cheapest route.

Algorithm outline:

1. Put start node in priority queue with $g = 0$.
2. Remove node with least $g(n)$; if it is the exit, return the path.
3. Otherwise expand it: for each neighbour (not a wall) compute $g = g(\text{parent}) + \text{step cost}$.
4. Insert new nodes, or update an existing frontier node if a cheaper path is found; keep an explored set.
5. Repeat until goal is found or the queue is empty (failure).

- **Properties:** Complete (if $\epsilon > 0$ and b is finite) and optimal (always returns least-cost path).
- **Time complexity:** $O(b^{(1+\lceil C^*/\epsilon \rceil)})$, which can be much larger than b^d .
- **Space complexity:** $O(b^{(1+\lceil C^*/\epsilon \rceil)})$ – it stores every generated node.
- **Advantages:** Guarantees the cheapest path even with unequal move costs (e.g. mud, slopes, turns); complete.
- **Disadvantages:** Explores in all directions equally, so many irrelevant cells are visited; exponential memory is the main bottleneck on a large maze; can be slow with tiny step costs.

2.3 Iterative Deepening Search (IDS)

Working: IDS repeatedly runs Depth-Limited DFS with limits 0, 1, 2, 3, ... until the goal is found. It combines the low memory of DFS with the completeness and shallow-goal optimality of BFS. The robot goes down one corridor to the current limit, backtracks, and tries other branches; if no exit is found, the limit is increased and the search restarts.

Algorithm outline:

```
for limit = 0,1,2,... :  
    result = DepthLimitedSearch(start, limit)  
    if result != cutoff: return result
```

- **Properties:** Complete (finite b); optimal when all step costs are equal (finds the shallowest exit).
- **Time complexity:** $O(b^d)$. Repeated work is small: nodes at depth d are generated once, those at depth $d-1$ twice, etc. Total $\approx (d)b + (d-1)b^2 + \dots + b^d$, dominated by the last level.
- **Space complexity:** $O(b \cdot d)$ – only the current path and siblings are stored (linear).
- **Advantages:** Very low memory, ideal for an embedded robot with limited RAM; finds the shortest (fewest-steps) path; no need to know the depth of the exit in advance.
- **Disadvantages:** Re-expands upper levels repeatedly; not optimal for non-uniform costs; without a visited check it can revisit cells in mazes with loops.

2.4 Comparison of UCS and IDS

Criterion	UCS	IDS
Expansion order	Lowest path cost $g(n)$	Depth-wise, increasing limit
Complete	Yes ($\epsilon > 0$)	Yes
Optimal	Yes – least cost, any step costs	Only for uniform step costs (shallowest)
Time	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	$O(b^d)$
Space	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ – exponential	$O(b \cdot d)$ – linear
Best when	Costs vary and memory is available	Memory is limited, depth unknown
Maze example	Cheapest route avoiding rough terrain	Fewest moves with small memory

2.5 Relating the problem to intelligent agent types

The maze environment is single-agent, static (maze does not change), discrete, deterministic (if the robot is accurate) and sequential, but initially partially observable because the robot only senses nearby walls. This decides how the agent must be designed:

- **Simple reflex robot:** Rules such as “wall ahead $\rightarrow$ turn right” (wall-following). It has no memory, so it can loop forever in mazes with cycles or islands; it is neither complete nor optimal.
- **Model-based robot:** Builds and updates a map of visited cells, walls and dead ends from its sensors. This handles partial observability and avoids revisiting cells; it is the necessary basis for any search in an unknown maze.
- **Goal-based robot:** Has the explicit goal “reach the exit” and uses search (BFS/IDS/UCS) on its map to produce an action sequence. Behaviour changes automatically when the goal changes.

- **Utility-based robot:** Evaluates paths by utility, e.g. $-(\text{distance} + \text{energy} + \text{risk} + \text{time})$. It can prefer a slightly longer but smoother and safer path; UCS is the natural search algorithm here because it minimises the path cost represented by utility.

Thus agent design determines decision-making: a reflex design makes immediate, local, memory-less decisions; a model-based design makes informed decisions using a map; goal-based design makes planned decisions; utility-based design makes optimal, trade-off-aware decisions. The search strategy is the “brain” that the goal- or utility-based agent uses to generate its plan.

2.6 Best combination and justification

Recommended: A model-based, goal-based agent (with utility-based cost awareness) using Iterative Deepening Search for uniform-cost mazes, and Uniform Cost Search when movement costs differ.

- **Unknown layout:** A model (map memory) is mandatory because the robot sees only locally; it prevents infinite loops and repeated exploration.
- **Memory constraint:** Robots have limited onboard memory; IDS needs only $O(b \cdot d)$ space whereas UCS/BFS may need exponential memory.
- **Exit depth unknown:** IDS needs no depth bound, yet is complete and returns the shortest path, with time $O(b^d)$ similar to BFS.
- **Different move costs:** If turning, slopes or surface types add cost, the utility-based element with UCS guarantees the least-cost route.
- **Practical enhancement:** Maintain an explored-cell set (graph search) to avoid repeated states and loops. If information about the exit later becomes available, switching to an informed algorithm such as A^* would further reduce time.

2.7 Conclusion

UCS provides optimality in cost at the price of exponential memory, while IDS provides completeness and shortest-path solutions with linear memory at the price of some repeated expansion. Combining a model-based, goal-based agent with IDS (or UCS for weighted moves) gives the most reliable and resource-efficient solution for a robot exploring an unknown maze.